1 - Il est nécessaire de définir explicitement une convention telle que big-endian car un tableau byte[] ne possède aucun typage, ce qui oblige de définir l'ordre des quatres octets d'un entier. Si writeInt utilisait big-endian et que readInt utilisait little-endian sur une autre machine alors cela inverserais l'ordre des octets à la lecture, produisant une valeur incorrect.

2 - Une opération telle que memory[offset] & 0xFF est importante lors de la reconstruction d'une valeur entière car en java le byte est signé ce qui provoque une extensions de signes lors des conversions. Le rôle de l'opérateur binaire sert à masquer, extraire ou encore effacer certaines bits.

3 - Tester uniquement readInt(writeInt(x)) == x est insufissant car deux erreurs dans l'écriture et la lecture peuvent se compenser. Pour sampler correctement les valeurs de x, il faut générer des valeurs aléatoires sur tout le domaine et y inclure des cas limites. L'inspection directe des octets permet de détecter davantage les classes d'erreurs car elle valide l'organisations physique en mémoire

4 - Un bitmap est plus compact qu'une représentation utilisant un entier par bloc car il utilise un seul bit par bloc au lieu de 32 bits pour représenter l'état d'allocation. Sa taille minimale est de N/8 octets pour un système de N blocs.

5 - Les structures du système de fichiers doivent occuper des zones déterministes car la mémoire brute simulée ne possède aucune structure propre, obligeant le système à calculer l'emplacement des métadonnées via des offsets fixes. Si la position du bitmap est changée sans mettre à jour ces offsets, les opérations d'allocation écriront au mauvais endroit, modifiant d'autres structures système et corrompant le système de fichiers.

6 - Séparer les métadonnées du contenu permet de conserver des structures de taille fixe. L'inode peut-être considéré comme une structure permettant de retrouver le contenu d'un fichier sans contenir directement le contenu grâce à des pointeurs qui indiquent les adresses des blocs de données sur le disque.

7 - allocateBlock() doit commencer la recherche à partir du bloc 129 car les blocs 0 à 128 sont strictement réservés aux structures internes du système telle que SuperBloc ou encore Bitmap. Si la recherche commençait à 0, l'allocateur pourrait attribuer un bloc réservé à un fichier utilisateur.

8 - Deux systèmes peuvent disposer du même nombre de blocs libres mais présenter des niveaux de fragmentation différents selon la disposition spatiale de ces blocs. Par exemple, un système avec 4 blocs libres contigus peut enregistrer un fichier en un seul morceau, alors qu'un système ayant 4 blocs libres séparés par des blocs occupés sera forcé de découper le même fichier en plusieurs fragments dispersés.